

📖 AI-Driven School 第4回講義

知っていれば 作れる・選べる人になる

漠然としたアイデアを、実際に動くツールに変える。その思考プロセスと、AI活用を深める3つの観点 ——

この講義の3つのパート

1 Anthropicブログの核心

AI活用の本質を一言で —— 「そのトークンはコンテキストを消費するに値するのか」

2 AI活用が上手になる3つの観点

聞き方・スキル磨き・ルール設定

3 アイデアを企画にする4つのステップ

分解 → 見極め → MVP → POC

📖 Part 1

AI活用の「聖書」 Anthropicブログの核心

Anthropic社が公開した公式エンジニアリングブログ『[Effective Context Engineering for AI Agents](#)』。AI活用のすべてはこの記事に書いてあることに収束されると言ってもいいほど重要な記事です。

📖 AI活用の本質を一言で

「そのトークンはコンテキストを消費するに値するのか」

AIに指示をする時に、少なすぎず多すぎない——特に多すぎるのは良くない。本当に必要な情報だけを与えることができるだろうか。これがAI活用の極意。

💡 今はわからなくても大丈夫

これはHowToというよりビジョン（指針）。自分でツールを作る中で、どこかのタイミングで腹落ちする瞬間が必ず来る。今は「そうなんだ」で置いておいて、常にこの問いが土台にあることだけ覚えておく。

🌟 Part 2

AI活用が上手になる3つの観点

講義を受けて課題に取り組んできた今だからこそ、知っておいてほしい3つのポイント。



観点1: わからない時のAIへの聞き方

何がわからないかを渡しやすくするのが最初



スクショ共有を効率化

画面キャプチャを一発でコピーできる環境を整えておく。手打ちより圧倒的に早い。

MacユーザーにはCleanShotがおすすめ



AIに噛み砕かせるフレーズ

「入社したての新卒社会人でもわかるように、専門用語は解説しながら説明して」

概念が複雑な時は「アスキーアートで直感的にわかるように図解して」を追加。



理解度をAIに確かめてもらう

「こういう理解で合っていますか」と自分の言葉で説明してフィードバックをもらう。人に説明できるレベルが「理解できた」の基準。



ウェブ検索をさせる

AIは学習時点で知識が止まる。最新情報が欲しい時は「検索して」と伝えるか、ウェブ検索が走っているか確認する。



観点2: 仕事を楽にする

スキルを磨いてコンテキストを整理するに尽きる

新しいエージェントを立ち上げるたびに、ゼロからプロジェクト情報を渡すのは非効率。だからコンテキストを整理してスキルを磨き続ける。



実例: Inside Stories（月額マガジン）の原稿スキル



1年以上かけて磨いてきた結果、そのまま読み上げても問題ないレベルに



観点3: 最初から全部わかっているAIに育てる

常に適用したいルールを設定する

毎回「新入社員でもわかるように」と伝えるのは面倒。**共通のルール設定**で、どんな時もAIが自分好みに振る舞うようにできる。

👤 ユーザールール

どんなプロジェクトでも常に適用。例: 「専門用語を避けて説明して」「Macを使っている」「オフィスは〇〇」

📁 プロジェクトルール

特定のリポジトリだけで適用。例: Cursor Rules、CLAUDE.md



注意: 優先順位がある。チャットの指示 > プロジェクトルール > ユーザールール。共通ルールは優先度が低いので、**迷ったら余計なことは書かない方がいい**。AIにとってノイズになる。

📖 Part 3

アイデアを企画にする4つのステップ

無人島に地図もコンパスもなく放り出された状態から、道を作るプロセス。

リアルタイム議事録ツールを実例に解説します。



課題を分解する

Step 1



意味を見極める

Step 2



MVPを定義する

Step 3



POCで検証する

Step 4



Step 1: 課題を分解する

「何を作ればいいかわからない」—— 多くの人が最初にぶつかる壁。その原因は、**物事を塊として見てしまっている**こと。分解すればアイデアが見えてくる。

アンラーニング = 無意識を意識に変えること



プロ野球選手

感覚でバットを振れる。初心者「どうやるの？」と聞かれると困る。無意識でやっていることを**言語化（＝分解）**しなければ教えられない。



会議に慣れた人

「会議は面倒だな」で止まっている。何が面倒なのかを**分解してみると**、見えていなかった課題が次々に出てくる。

 良いアイデアを出すとは、自分が無意識でやっていることを意識すること。

「会議は面倒だな」を分解してみる

漠然とした1つの不満が、9つの具体的な面倒に変わる

1 聞き取れなかったことを確認する面倒

2 誰がいつ何を言ったか思い出す面倒

3 テキストに起こす面倒

4 要点を整理する面倒

5 タスクを抜き出す面倒

6 「なるはやで」を「4/15まで」に変換する面倒

7 決定が覆った時に修正する面倒

8 タスク管理ツールに登録する面倒

9 関係者に共有する面倒



分解のコツ: AIに聞き出してもらう

「私が普段やっている会議の流れの解像度を上げたいので、最初から最後まで全部聞き出してほしい。質問は**1つずつ**お願いします」

自分で書き出すとつい省略してしまう。AIに聞かれると省略できない。「何も言わないと一気に質問をぶつけられるので、**1つずつ聞いて**」がポイント。

🔍 Step 2: 自分が作る意味を見極める

課題を分解してアイデアが出たら、**同じことができるものが世の中にないか**を調べる。ポイントは**分解した後に**調べる。分解前に調べても、自分が何に困っているかわからないから良し悪しを見極められない。

既存サービスでカバーできる範囲を見極める

✅ 既存で解決できる

- ✓ テキストに起こす
- ✓ 要点を整理する
- ✓ タスクを抜き出す（精度は課題あり）

⊕ 自分が作る価値がある

- ☆ リアルタイムにToDoが出る
- ☆ 曖昧な表現を自動で具体化
- ☆ 自社の会議に最適化

⚠️ 「誰も思いついていなかった」はあり得ない

アイデアは必ず思いつかれている。何かしらの**ネック**があって実現できないか、優先順位が下がっていると考えべき。

💰 コストのネック

リアルタイム処理はAPIの使用量が多く、顧客に一定料金内で提供するにはコストが高すぎる

🛡️ プロダクトリスク

会議途中の曖昧な段階でToDoを表示すると間違える可能性が高く、商用プロダクトとしてリリースが難しい

💡 自分用なら利益を上乗せする必要がなく、自社に最適化するだけでいいのでハードルが下がる。

✂️ Step 3: MVPを定義する

MVP（Minimum Viable Product＝必要最小限の機能のプロダクト）。何か作り出す人は夢に溢れていて、つい機能を詰め込んでしまう。本当に必要な機能だけを見極めて、価値があるかを検証しようというスタンス。

⚠ MVP開発の最重要マインド

コードを書いたら負け

コードを書かずに同じような体験を作れないかと考える。ツール開発の最大の悲劇は、誰も使わないものを作ってしまうこと。

MVPで検証するのは2つの仮説

❓ 課題仮説

「会議で決まったことを後で思い出して共有するのが面倒のはずだ」

💡 解決仮説

「リアルタイムにToDoが可視化されれば、会議が楽になるはずだ」



コードを書かない検証の例

会議にオンラインでメンバーを1人参加させ、手打ちでリアルタイムにToDoをまとめてもらう。これだけでリアルタイムAI議事録ツールと同じ体験ができる。

他のメンバーが「これはいいね、また使いたい」と言わなければ、頑張ってコードを書いてもツールを作る意味はなかった。

⊖ MVPの基準：「恥ずかしい」と感じるレベル

- ✗ 誰もがアクセスできるウェブアプリ → 自分のパソコンだけで動く状態で十分
- ✗ ブラウザで見えるおしゃべり → マークダウンファイルで十分
- ✗ メンバーを自由に登録できる機能 → 設定ファイルに書けばいい

「やりたいことがだいたいできた」と思ったらやりすぎ。削って削ってみすばらしいけど、仮説を検証できるラインを見つける。



Step 4: POCで検証する

POC（Proof of Concept＝概念実証）。自分たちのやりたいことが、既存の技術で本当にできるのかを検証する。MVPが「使う人が価値を感じるか」の検証なら、POCは「**技術的にできるのか**」の検証。

処理の流れを見て、怪しいところから検証する

できるとわかっていることは後回し。一気に全部作らない。



マイクを起動して一定間隔で音声を保存

POC必要

プログラムからマイクを起動できるのか？一定間隔で区切れるのか？



音声を文字起こし

できる

ChatGPT / Claude / Gemini のAPIで音声を送れば文字起こしできる



文字起こしからToDo抽出

できる

文章をAIに渡してToDoを出してもらうのはブラウザのAIでもできる



マークダウンファイルに反映

できる

ファイルの更新は確実にできる。検証不要。



わからないことはAIに聞いて進める

10年前ならテクノロジーリテラシーがないと詰まっていた。今はAIと対話しながら、自分で作りたいツールの処理の流れを理解できる。**これは革命的なこと。**

実際にAIと対話する中で、リアルタイム文字起こしに強い「Deepgram」というサービスの存在を知り、採用に至った。最初から全部わかる人はいない。



まとめ——第4回講義で覚えておきたいこと

01 AI活用の本質は「必要な情報だけを渡す」こと

Anthropic社のブログが伝えるメッセージを一言でまとめるなら「そのトークンはコンテキストを消費するに値するのか」。少なすぎず多すぎない、特に多すぎるのは良くない。今はわからなくても、ツールを作る中で腹落ちする瞬間が来る。

02 AI活用は3つの観点で深まる

わからない時の聞き方を磨く。スキルとコンテキストを育てて仕事を楽にする。共通ルールで最初から自分を理解しているAIに育てる。日々の積み重ねが品質を変える。

03 アイデアは「分解」から始まる

何を作ればいいかわからないのは、物事を塊として見ているから。無意識でやっていることを分解（アンラーニング）すると、課題が見えてアイデアが生まれる。AIに1つずつ質問してもらうのが効果的。

04 コードを書く前にやるべきことがある

自分が作る意味を見極め、MVPは恥ずかしいレベルまで削る。POCでは怪しいところから1つずつ検証する。一気に全部作らない。このプロセスはすべてのツール開発で使える。



次回講義: 自分で作った報告ツールの発表会

MVPでいいので自動報告ツールの完成を目指そう。うまく動かなかったとしても、それも含めて貴重な学び。

今回の4ステップを活用して:

- 1 自分の課題を**分解**してみる
- 2 自分が作る**意味があるか**を確認する
- 3 **恥ずかしいレベル**まで削ったMVPを定義する
- 4 怪しいところから**1つずつ**検証する

AI-Driven School の図解ツールで作成